

StaleyNET: Text-Conditioned Music Generation Trained from Scratch on Edge GPUs in Rust

Arpan Pathak

November 2026

Abstract

This paper describes StaleyNET, a small system that generates music from a short text description. We built it from first principles, mainly as a way of understanding what each part of such a system contributes. All components, including the GPU kernels, are written in Rust, and the models are trained from random initialisation on two NVIDIA Jetson devices with about 23 hours of Creative Commons music. Sound is represented by quantised log-mel spectrograms, and a decoder-only Transformer learns to predict these tokens from a caption and, optionally, from lyrics. Because every stage is our own, we could examine the stages one at a time. These measurements suggest that the quality of the generated audio is limited mainly by the tokenizer rather than by the language model: a model trained on a single excerpt reproduces it exactly, while the tokenizer on its own loses much of the spectral detail. Guided by this observation, we revised the tokenizer and roughly halved its reconstruction error. We also describe a simple paired test for whether a model makes use of the lyrics it is given; our first lyrics-conditioned models do not yet pass it. Each algorithm is presented with a small worked example, in the hope that the paper is useful to readers who would like to build a similar system.

1 Introduction

Generating music from a sentence of text has made remarkable progress in recent years [1, 3]. Most current systems combine several large components that are trained separately: a neural codec that turns audio into tokens, a text encoder that turns the prompt into vectors, and a Transformer that relates the two. This division of labour works very well, although it can make it difficult to tell which component is responsible for a particular quality, or a particular flaw, in what one hears.

In this work we approached the problem from the other direction and built a complete, if modest, system ourselves (Figure 1). StaleyNET computes its spectrograms, learns its tokenizer, trains its Transformer and reconstructs its waveforms without any pretrained component. It is written in Rust, and its GPU kernels are compiled from Rust by cuda-oxide [19]. We trained it on an NVIDIA Jetson Orin NX with 16 GB of memory and on an NVIDIA Jetson AGX Thor with 128 GB, two embedded computers designed for robotics. Owning every stage gave us the

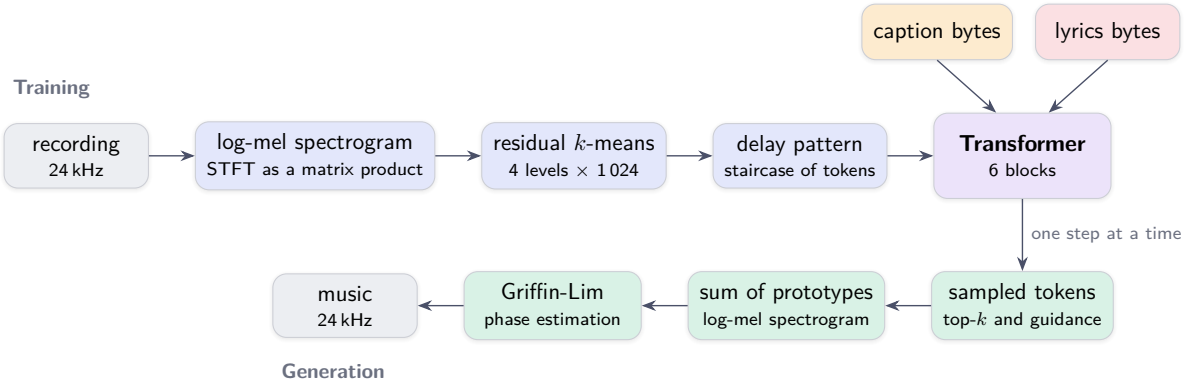


Figure 1: An overview of StaleyNET. During training (top row), a recording is converted into a staircase of discrete tokens, which the Transformer learns to predict from the caption and the lyrics. During generation (bottom row), the Transformer produces tokens one step at a time, and the tokens are turned back into sound.

opportunity to study each one in isolation, and much of the paper is about what that study revealed.

Our first models learned text-conditioned structure well above chance, yet the voices they produced sounded blurred. Two experiments helped us locate the cause. When we passed held-out recordings through the tokenizer and the vocoder alone, without any model, most of the spectral detail was already lost. When we trained the model on a single excerpt, it reproduced that excerpt token for token. Together these results suggested that the tokenizer was the main limitation, and a revised tokenizer that quantises groups of frequency bands separately reduced the reconstruction error by about half.

The contributions of this paper are as follows.

- A complete text-to-music pipeline written in Rust, including its GPU kernels, and trained from scratch on embedded GPUs.
- A description of each algorithm with a worked example and, where an algorithm is easy to get wrong, an argument for its correctness.
- A way of adding a lyrics input to a trained model without discarding what it has learned, together with a paired test of whether the input is used.
- A stage-by-stage analysis that separates the errors of the tokenizer, the vocoder and the language model, and a tokenizer study guided by it.

We write algorithms in the style of Cormen et al. [4], as in our earlier tutorial on tiled matrix multiplication [20]. Each procedure first states its inputs and its output in words; its lines are numbered, and the text refers to them. The range $[a, b)$ denotes the integers $a, \ldots, b-1$, indices start at 0, and losses are measured in nats.

2 Related work

AudioLM [2] showed that audio can be generated by a language model over discrete tokens. MusicLM [1] conditioned this approach on text, and MusicGen [3] simplified it to a single Transformer over the codebooks of a neural codec, interleaved by a delay pattern. We follow MusicGen in using the delay pattern, top-k sampling [9] and classifier-free guidance [12], and

we replace its pretrained components with k-means quantisation of log-mel frames and with raw text bytes, as studied for text by ByT5 [31].

Jukebox [7] generated singing from lyrics and learned the correspondence between words and sound from 1.2 million songs. Singing synthesis systems such as DiffSinger [16] instead work from a musical score with phoneme durations. Our lyrics experiments rely on the line-level timings of the JamendoLyrics benchmark [27, 8]. Neural codecs [32, 6, 15] and neural vocoders [14, 24] achieve far higher fidelity than k-means tokens and the Griffin-Lim algorithm [10, 21], at a training cost beyond our setting; Section 9.1 measures what our simpler choices cost.

3 From sound to spectrogram and back

A waveform sampled at 24 kHz contains 24 000 numbers per second, and much of that information describes the phase of each vibration, to which the ear is not very sensitive. We therefore describe sound by its log-mel spectrogram: the energy in 128 frequency bands spaced according to human pitch perception, measured about 94 times per second and compressed logarithmically.

3.1 Resampling

The training recordings arrive at several sample rates, most often 44.1 kHz, and we convert them all to 24 kHz. Lowering the rate requires first removing all content above the new Nyquist frequency of 12 kHz; otherwise that content folds back into the audible range, an effect known as aliasing.

Definition 3.1 (Bandlimited resampling)**.** Given samples x_m at rate f_{in}, the resampled signal at rate f_{out} is

$$y_n \;=\; \sum_{m} x_m \, h(t_n - m)\,, \qquad t_n = n\,\frac{f_{\text{in}}}{f_{\text{out}}},$$

where h is a low-pass filter. Each output sample is a weighted sum of the input samples near the fractional position t_n.

We use a windowed sinc filter, $h(t) = c\,\operatorname{sinc}(ct)\,w(ct/Z)$, in which c is the cutoff relative to the input rate, $Z = 24$ is the number of zero crossings kept on each side, and w is a Kaiser window with $\beta = 8.6$ [26]. Evaluating h anew for every output sample would be slow, but fortunately the fractional part of t_n takes only a few values.

A worked example. For a conversion from 44 100 Hz to 24 000 Hz the two rates share the factor 300, so $t_n = 147n/80$. For $n = 1$ we obtain $t_1 = 1 + 67/80$. Every t_n has a fraction of the form $p/80$ with $p \in [0, 80)$, so exactly 80 sets of filter weights are needed. These sets are called the *phases* of a polyphase filter, and they are computed once.

RESAMPLE$(x, f_{\text{in}}, f_{\text{out}})$

Input: the samples $x_0, x_1, \ldots$ of a recording, its sample rate f_{in}, and the desired rate f_{out}, both in Hz.
Output: the samples $y_0, y_1, \ldots$ of the same sound at rate f_{out}.

```
1   g = gcd(f_in, f_out)
2   L = f_out / g   // number of phases
3   M = f_in / g   // input step per output sample, in L-ths
4   c = 0.97 · min(1, f_out / f_in)   // cutoff
5   R = ⌈Z / c⌉   // filter half-width
6   for p ∈ [0, L) {
7       for j ∈ [−R, R] {
8           h_p[j] = c sinc(c (j − p/L)) w(c (j − p/L) / Z)
9       }
10  }
11  for each output index n {
12      m = ⌊nM / L⌋   // whole part of t_n
13      p = nM mod L   // fraction of t_n
14      y_n = Σ_{j∈[−R,R]} h_p[j] x_{m+j}
15  }
```

Lines 1–5 determine the geometry of the filter, lines 6–10 compute the L phases once, and line 14 is the single dot product needed for each output sample. The factor 0.97 in line 4 keeps the transition band of the filter just below the new Nyquist frequency. A test confirms that the filter does its job: a 15 kHz tone is attenuated by more than 80 dB when 48 kHz audio is converted to 24 kHz.

3.2 The spectrogram as a matrix product

The signal is divided into frames of $N = 1024$ samples, taken every $H = 256$ samples, and each frame is multiplied by a Hann window. The strength of each frequency in each frame is usually computed with the fast Fourier transform. We computed it instead with a matrix product, so that the spectrogram runs on the same GPU kernel as the Transformer and benefits from the same tests (Section 8).

Definition 3.2 (Log-mel spectrogram)**.** Let row f of the matrix F hold frame f of the signal, and let W have entries $W_{nk} = w[n]\, e^{-2\pi i k n / N}$ for $k \in [0, N/2]$, where w is the Hann window. The log-mel spectrogram is

$$S \;=\; \ln \max\big(|FW|^2 B,\; 10^{-5}\big),$$

where the square is taken entry by entry and the columns of B are 128 triangular mel filters [25].

Column k of W holds a windowed sine and cosine at $k \cdot 23.4$ Hz, so row f of FW measures how much of each frequency frame f contains. The real and imaginary parts of W are two real matrices, and B adds a third product. This formulation needs about $N / \log_2 N \approx 100$ times the arithmetic of an FFT. Even so, on the Orin NX it analyses 60 s of audio in 47 ms, against 73 ms for a multithreaded CPU FFT (Table 8), and it agrees with the FFT to within 10^{-2} log units.

3.3 Estimating the phase

A generated spectrogram tells us how strong each frequency is, but not its phase. The Griffin-Lim algorithm [10] estimates a phase by alternating between two requirements: the magnitudes should match the target, and the spectrogram should be that of an actual waveform.

GRIFFIN-LIM($|S|, T$)

Input: the target magnitudes $|S|$, one row per frame and one column per frequency, and the number of rounds T.
Output: a waveform x whose spectrogram approximates $|S|$.

1 $X = |S| \cdot e^{i\phi}$ with random phases ϕ
2 $x = \mathrm{ISTFT}(X)$
3 $P_{\mathrm{prev}} = \mathrm{STFT}(x)$
4 **for** $t \in [0, T)$ {
5 $P = \mathrm{STFT}(x)$ *// nearest consistent spectrogram*
6 $A = P + \alpha\,(P - P_{\mathrm{prev}})$ *// momentum*
7 $X = |S| \cdot A/|A|$ *// keep the phase, restore the magnitude*
8 $P_{\mathrm{prev}} = P$
9 $x = \mathrm{ISTFT}(X)$
10 }
11 **return** x

Here STFT and ISTFT denote the short-time Fourier transform and its inverse. Line 5 replaces the current estimate by the spectrogram of a real waveform, and line 7 restores the target magnitudes. Line 6 is the acceleration proposed by Perraudin et al. [21], which extrapolates along the last change with momentum $\alpha = 0.99$; in the first round it has no effect, because $P = P_{\mathrm{prev}}$. We use $T = 64$ rounds, after mapping the 128 mel bands back to 513 frequencies with a regularised least-squares inverse of B.

4 From spectrogram to tokens

A language model chooses each next symbol from a finite vocabulary, whereas a spectrogram consists of real numbers. The tokenizer bridges this gap. Each band of the spectrogram is first standardised with its mean and standard deviation over the training set, so that quiet and loud bands carry similar weight. A *step* is then formed from consecutive frames; in our first tokenizer a step joins 2 frames, giving 256 numbers that cover 21.3 ms.

4.1 Residual quantisation

One can think of residual quantisation as describing a colour first by its family, then by its shade within the family, and then by finer and finer corrections. Each level names the closest of K prototypes to whatever the previous levels have left unexplained.

Definition 4.1 (Residual vector quantisation)**.** Let $E_0, \dots, E_{C-1}$ be codebooks, each a list of K prototype vectors. A step x is encoded as the indices $c_0, \dots, c_{C-1}$ defined by

$$r_0 = x, \qquad c_\ell = \arg\min_{k \in [0,K)} \|r_\ell - E_\ell[k]\|^2,$$
$$r_{\ell+1} = r_\ell - E_\ell[c_\ell],$$

and it is decoded as $\hat{x} = \sum_\ell E_\ell[c_\ell]$. The vector r_ℓ is the *residual*: the part of x that levels $0, \dots, \ell - 1$ have not yet described.

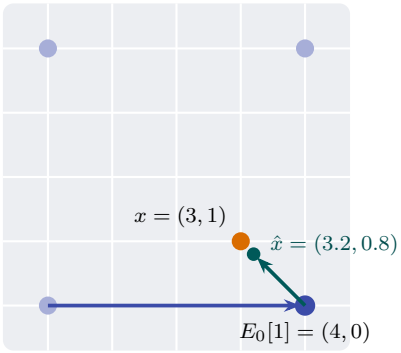


Figure 2: Residual quantisation of $x = (3, 1)$ in two levels. Level 0 (blue) chooses the prototype $(4, 0)$, and level 1 (green) adds the correction $(-0.8, 0.8)$.

A worked example. In two dimensions, consider the codebooks

$$E_0 = \{(0,0),\ (4,0),\ (0,4),\ (4,4)\},$$
$$E_1 = \{(0.8,0.8),\ (-0.8,0.8),\ (-0.8,-0.8),\ (0.8,-0.8)\},$$

and the point $x = (3, 1)$ (Figure 2). The squared distances from x to the prototypes of E_0 are $10, 2, 18, 10$, so $c_0 = 1$ and $r_1 = (-1, 1)$. The closest entry of E_1 to $(-1, 1)$ is $(-0.8, 0.8)$, so $c_1 = 1$ and $r_2 = (-0.2, 0.2)$. The error falls from $\sqrt{2} \approx 1.41$ after one level to $\sqrt{0.08} \approx 0.28$ after two.

4.2 Training the codebooks

TRAIN-CODEBOOKS(X, C, K, I)

Input: a matrix X whose rows are training steps, the number of levels C, the number of prototypes per level K, and the number of rounds I.
Output: the codebooks $E_0, \dots, E_{C-1}$.

```
1   R = X // what remains to be described
2   for ℓ ∈ [0, C) {
3     E_ℓ = K rows of R, chosen at random
4     repeat I times {
5       a_i = NEAREST(R_i, E_ℓ) for each row i
6       E_ℓ[k] = mean{R_i : a_i = k} for each k
7       if no row has a_i = k
8         E_ℓ[k] = a random row of R
9     }
10    R_i = R_i − E_ℓ[NEAREST(R_i, E_ℓ)] for each row i
11  }
```

NEAREST(r, E) returns the index of the prototype in E closest to r. Lines 5–6 are the two moves of Lloyd's algorithm [17]: each row is assigned to its nearest prototype, and each prototype then moves to the mean of its rows. Lines 7–8 restart a prototype to which no row was assigned, so that none of the K entries goes unused, and line 10 passes the residuals on to the next level. We use $I = 20$ rounds on 200 000 steps drawn from the training set.

4.3 The nearest prototype as a matrix product

Lemma 4.1. *For a vector x and prototypes $e_0, \dots, e_{K-1}$, $\arg\min_k \|x - e_k\|^2 = \arg\min_k \left(\|e_k\|^2 - 2\, x \cdot e_k\right)$.*

Proof. Expanding gives $\|x - e_k\|^2 = \|x\|^2 - 2\, x \cdot e_k + \|e_k\|^2$, and the first term does not depend on k. □

In the example above, the squared norms $\|e_k\|^2$ are $0, 16, 16, 32$ and the dot products $x \cdot e_k$ are $0, 12, 4, 16$, so the scores are $0, -8, 8, 0$, and the smallest again selects $(4, 0)$. On the GPU the dot products of 8 192 steps with 1 024 prototypes form a single matrix product, after which a small kernel finds the smallest score in each row.

4.4 Quantising groups of bands

The analysis of Section 9.1 led us to a refinement. Instead of quantising a whole step at once, we divide its 128 bands into G equal groups and give each group its own residual codebooks. With one frame per step and $G = 4$, each group is a vector of 32 numbers, which 1 024 prototypes can describe more finely than a whole step. In our experiments the grouping helped somewhat less than we expected, and the additional codes helped more (Appendix C). The codes of a step are ordered level by level: first the coarse codes of all G groups, then their first corrections, and so on, so that the coarse-to-fine order relied on by the delay pattern (Section 5) is preserved.

With $G = 4$ groups and 4 levels each, a step is described by 16 codes. Table 9 in the appendix compares this and other configurations.

5 The model

The model (Figure 3) is a decoder-only Transformer [28, 23]. It reads text and audio as a single sequence and predicts, at each position, what comes next, much as a text model predicts the next word.

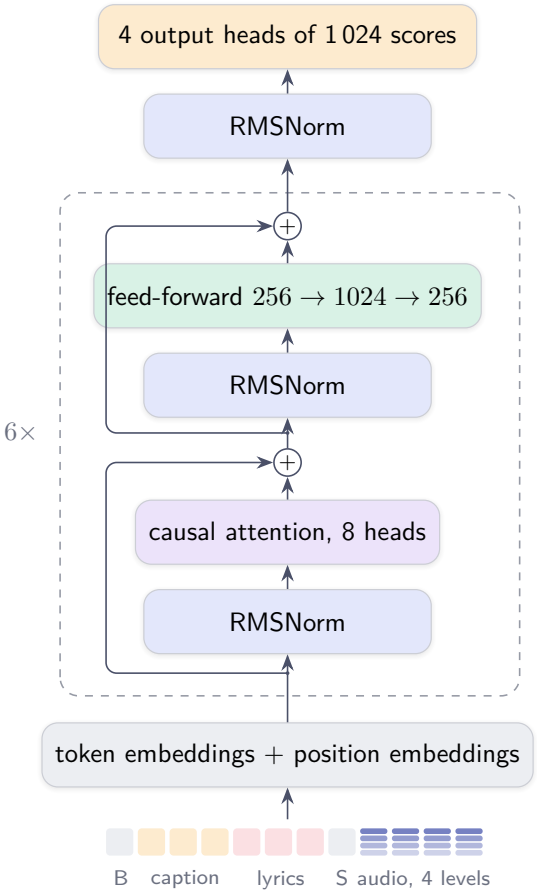


Figure 3: The model. Text bytes and audio tokens share one sequence, and each audio position carries all the codes of one step. A stack of pre-normalised blocks, each adding its result to its input, ends in one output head per code.

5.1 One sequence for text and audio

The vocabulary contains the 256 possible byte values [31], three markers (`BEGIN`, `SEPARATOR` and `PADDING`) and $C \cdot K$ audio symbols, one for each prototype of each codebook. The text occupies a fixed 128 positions, namely `BEGIN`, 126 caption bytes and `SEPARATOR`, so that the audio always begins at the same position. An audio position carries all C codes of a step, and its input vector is the sum of their embeddings and the embedding of the position.

5.2 The delay pattern

At each position the model predicts all C codes of a step. A finer code, however, corrects a coarser one and is easier to predict once the coarser one is known.

Definition 5.1 (Delay pattern [3])**.** Position t of the delayed sequence holds, in slot ℓ, the code $a^{\ell}_{t-\ell}$ of codebook ℓ for frame $t-\ell$, and an empty marker when $t-\ell$ is not a frame. A piece of F frames becomes $F+C-1$ positions.

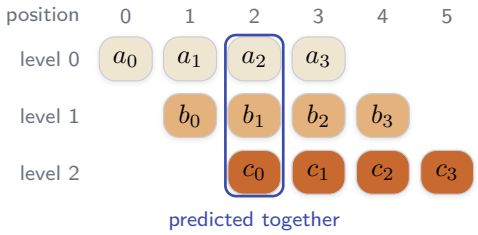


Figure 4: The delay pattern for three codebooks and four frames. At position 2 the model predicts a_2, b_1 and c_0, and the coarser codes that they refine are already part of its input.

In Figure 4, position 2 predicts a_2, b_1 and c_0 together. By then a_1, which b_1 refines, and a_0 and b_0, which c_0 refines, are already known. The price is $C-1$ additional positions, whereas giving each codebook its own positions would multiply the length of the sequence by C.

5.3 The network and its objective

In each block, attention lets every position gather information from itself and from earlier positions, and a small feed-forward network then transforms each position on its own. Both parts normalise their input [33, 30] and add their result to it:

$$h = x + \mathrm{Attention}(\mathrm{Norm}(x)),$$
$$y = h + \mathrm{FeedForward}(\mathrm{Norm}(h)),$$

where x is the sequence entering the block and y the sequence leaving it. The feed-forward network uses the GELU activation [11]. Our first model has 6 blocks of width 256 with 8 attention heads (7.1 million parameters); the larger model of Section 9.3 has 8 blocks of width 512 (42.9 million parameters).

At position p the model outputs, for each codebook ℓ, a probability distribution over the code $y_{p,\ell}$ that appears at position $p+1$. The loss is the cross-entropy, averaged first over the set V_ℓ of positions that have a target in codebook ℓ and then over the codebooks:

$$\mathcal{L} = \frac{1}{C}\sum_{\ell=0}^{C-1}\frac{1}{|V_\ell|}\sum_{p\in V_\ell} -\ln P_\theta(y_{p,\ell} \mid z_0,\dots,z_p),$$

where z_p is the input at position p and θ denotes the parameters. A model that assigns equal probability to all 1 024 codes has a loss of $\ln 1024 \approx 6.93$.

5.4 Gradients by hand

As far as we know, no automatic differentiation is available for Rust on the GPU, so each layer implements its own gradient. Attention is the least obvious case. For one head, let the rows

of Q, K and V be the queries, keys and values of the positions, and let d be the width of the head. Then

$$S = QK^\top/\sqrt{d} + M, \qquad P = \mathrm{softmax}(S), \qquad O = PV,$$

where the mask M is $-\infty$ above the diagonal, so that no position attends to a later one, and the softmax normalises each row.

Proposition 5.1. *Let ∂O be the gradient of the loss with respect to O, and similarly for the other matrices. Then $\partial V = P^\top \partial O$, $\partial P = \partial O\, V^\top$, $\partial S_{ik} = P_{ik}(\partial P_{ik} - \rho_i)$ with $\rho_i = \sum_j \partial P_{ij} P_{ij}$, $\partial Q = \partial S\, K/\sqrt{d}$ and $\partial K = \partial S^\top Q/\sqrt{d}$.*

Proof. The first two identities follow from $O = PV$. Within a row, $\partial P_{ij}/\partial S_{ik} = P_{ij}(\delta_{jk} - P_{ik})$, so $\partial S_{ik} = \sum_j \partial P_{ij} P_{ij}(\delta_{jk} - P_{ik}) = P_{ik}(\partial P_{ik} - \rho_i)$. The last two follow from $S = QK^\top/\sqrt{d}$. Where $M = -\infty$ we have $P_{ik} = 0$ and therefore $\partial S_{ik} = 0$. □

Apart from ∂S, every gradient is a matrix product and runs on the same kernel as the forward pass. Because hand-written gradients are easy to get wrong, a test perturbs three random entries of every parameter tensor by $\pm 10^{-2}$ and checks that the resulting change of the loss agrees with the computed gradient to within 5 %.

6 Lyrics

A caption describes how a piece should sound, but not which words should be sung. To make the words controllable, the model needs a second text input, and it needs training examples in which it can hear which words are sung where.

6.1 A position map that preserves a trained model

The lyrics occupy $L = 256$ byte positions between the caption and the separator. Each position p takes its position embedding from row $\pi(p)$ of a table, and the choice of π determines whether a model trained without lyrics can later be extended (Figure 5).

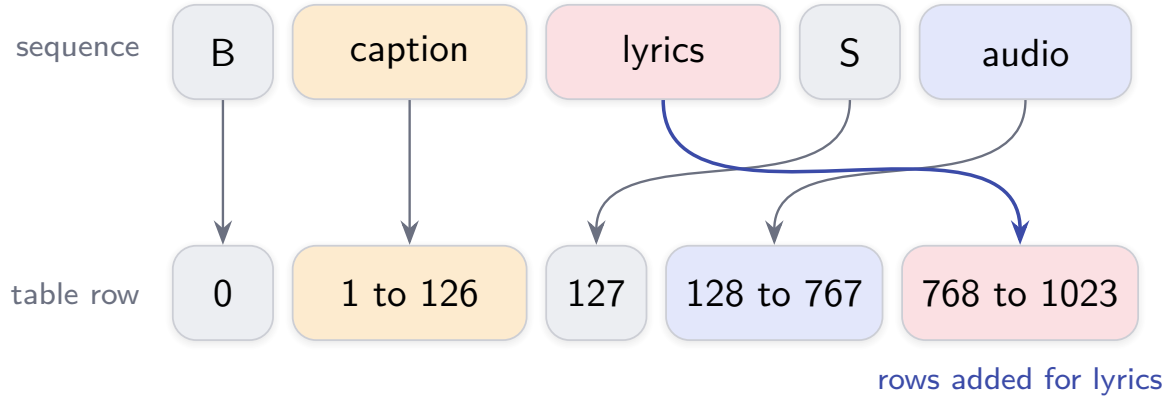


Figure 5: The position map π. Positions outside the lyrics keep the rows that they used before the lyrics were added, and the lyrics use new rows appended to the table.

Definition 6.1 (Position map)**.** Suppose the caption occupies positions $[1, 1{+}P)$, the lyrics occupy $[1{+}P, 1{+}P{+}L)$, and the model without lyrics has T rows. Then

$$\pi(p) = \begin{cases} p & \text{if } p < 1 + P, \\ T + p - 1 - P & \text{if } 1 + P \le p < 1 + P + L, \\ p - L & \text{otherwise.} \end{cases}$$

Lemma 6.1. *Every position outside the lyrics uses the row that the same part of the sequence used without lyrics, and the lyrics use only the new rows* $[T, T + L)$*.*

Proof. Caption positions are unchanged. The separator and the audio are shifted right by L positions, and π subtracts L. The lyrics are mapped onto $T + [0, L)$. □

For example, with $P = 2$, $L = 3$ and $T = 16$, positions 0 to 8 use the rows

$$0,\ 1,\ 2,\ 16,\ 17,\ 18,\ 3,\ 4,\ 5.$$

A trained model can therefore be extended by appending L new rows to its table. All learned parameters keep their meaning, and training simply continues.

6.2 Pairing audio with the words sung in it

A training example covers a window of a song, and it should carry the lines sung in that window. JamendoLyrics [8] gives the start and end time of every line of 78 songs in English, French, German and Spanish, and a window receives each line that overlaps it. Takes recorded with our studio come with their text but without timings, so we assume that their lines are spread evenly over the take; this is a fair approximation for short takes of continuous singing. Because the lyrics corpus is small (4.7 hours, against 23 hours of captioned music), each dataset receives a sampling weight: a weight of w makes each of its songs w times as likely to be drawn.

6.3 Testing whether the lyrics are used

A model can lower its loss on the lyrics corpus simply by learning how those particular songs sound, without reading their words. We therefore test the use of the lyrics directly, by giving the model the same audio with the correct words, with no words, and with words from another song.

LYRICS-TEST$(\theta, \mathcal{D}, B)$

Input: a trained model with parameters θ, a set $\mathcal{D}$ of held-out songs with lyrics, and a number of batches B.
Output: the mean paired difference $\bar{\Delta}$, its standard error s, and whether the difference is clearly positive.

1 **for** $b \in [0, B)$ {
2 draw a batch of windows from the songs in $\mathcal{D}$
3 $\mathcal{L}_b^{\text{right}}$ = loss of θ with each window's own lyrics
4 $\mathcal{L}_b^{\text{none}}$ = loss of θ with empty lyrics
5 $\mathcal{L}_b^{\text{wrong}}$ = loss of θ with lines from another song
6 }
7 $\Delta_b = \mathcal{L}_b^{\text{wrong}} - \mathcal{L}_b^{\text{right}}$ for each b
8 $\bar{\Delta} = \frac{1}{B} \sum_b \Delta_b$
9 $s = \sqrt{\text{Var}(\Delta)/B}$
10 **return** $\bar{\Delta}$, s, and whether $\bar{\Delta} > 2s$

Lines 3–5 score the same windows, so the differences in line 7 compare like with like, and most of the variation between windows cancels. A model that ignores the words gives $\bar{\Delta} \approx 0$. A model that reads them should predict the audio better with the right words than with the wrong ones, giving $\bar{\Delta} > 0$, measured in nats per code. The wrong lines in line 5 always begin at a sung line rather than at silence, and comparing lines 4 and 5 helps distinguish a reaction to the presence of text from a reaction to its content.

7 Generation

Generation repeats a single step: the model predicts a distribution for each codebook at the next position, one code is drawn from each, and the drawn codes are appended to the sequence.

7.1 The key/value cache

Position p attends to the keys and values of positions $0, \ldots, p$. These do not change once they have been computed, because no position attends forward, so a cache can keep them [22].

DECODE-STEP(z, p, cache)

Input: the codes z to place at position p, the position p itself, and the keys and values of positions $0, \ldots, p-1$ for every block.
Output: the scores of every codebook for position $p+1$; the cache is extended by position p.

1 $h = \text{embed}(z) + \text{position}(\pi(p))$
2 **for** each block j {
3 q, k, v = query, key and value projections of $\text{Norm}(h)$
4 $\mathit{cache}_j[p] = (k, v)$
5 $h = h + \text{Attend}(q,\ \mathit{cache}_j[0\,..\,p])$
6 $h = h + \text{FeedForward}(\text{Norm}(h))$
7 }
8 **return** the output-head scores of $\text{Norm}(h)$

Theorem 7.1. *Feeding a sequence position by position through* **DECODE-STEP** *yields, at every position, the same scores as a single forward pass over the whole sequence.*

Proof. We argue by induction over the blocks. At block 0, the input at position p depends only on the token and the position of p. Suppose that the inputs of block j agree at all positions up to p. Then the keys and values computed in line 3 agree, and when position p is processed the cache holds exactly those of positions $0, \ldots, p$. In the full pass the mask gives zero weight to later positions, so attention at p combines the same keys and values with the same weights, and the feed-forward network acts on each position separately. The outputs of block j, which are the inputs of block $j + 1$, therefore agree up to position p. □

A test checks the theorem numerically, to within 10^{-4} at every position, for models with and without lyrics; the second case also checks the position map π. Because the cache has a fixed capacity, a step allocates no memory.

7.2 Sampling and guidance

For each codebook we keep the $k = 250$ most probable codes and draw among them [9]. Because the text was removed from 10 % of the training examples, the same model can predict both with the text, giving scores z^{text}, and without it, giving z^{none}. Classifier-free guidance [12] amplifies their difference:

$$z = z^{\text{none}} + s\,(z^{\text{text}} - z^{\text{none}}),$$

where s is the guidance scale. For instance, if two codes score $(2, 1)$ with the text and $(1, 1)$ without it, the first has probability 0.73; with $s = 3$ the scores become $(4, 1)$, and its probability rises to 0.95. We use $s = 3$, which doubles the computation per step. Lyrics longer than L bytes are cut at line breaks into windows that are presented in turn, each over an equal share of the output, while the most recent audio is kept as context.

8 GPU kernels

Matrix products account for almost all of the arithmetic: the spectrogram, the nearest-prototype search, the projections of the Transformer and both products inside attention. Our tutorial [20] derives the shared-memory tiled product in detail; here we describe one further step, register blocking, and the way we made one gradient repeatable.

8.1 Register blocking

In the tiled kernel, a block of 16×16 threads copies tiles of A and B into shared memory, which reduces traffic to global memory sixteenfold [20, 13]. Every multiply-add still reads two values from shared memory, however, and shared memory is slower than a thread's registers. Register blocking [29] lets each thread compute an $r \times r$ group of outputs, so that every value it reads is used r times (Figure 6).

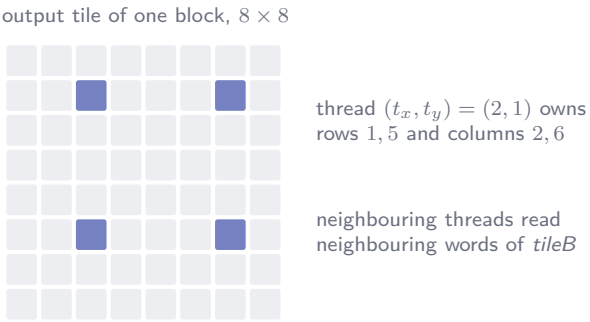


Figure 6: Register blocking with 4×4 threads and $r = 2$. Each thread computes r^2 outputs spread over the tile, in rows $t_y + 4i$ and columns $t_x + 4j$. Our kernel uses 16×16 threads, $r = 4$ and a 64×64 tile.

BLOCKED-THREAD(t_x, t_y)

Input: the coordinates (t_x, t_y) of a thread within a block of 16×16 threads; the block computes a 64×64 tile of $C = AB$, with $r = 4$.
Output: the 16 entries of the tile owned by the thread, written to C.

1 $sum[i][j] = 0$ for $i, j \in [0, r)$
2 **for** each phase t {
3 copy a 64×16 tile of A into *tileA*
4 copy a 16×64 tile of B into *tileB*
5 **BARRIER**()
6 **for** $u \in [0, 16)$ {
7 $\alpha_i = tileA[t_y + 16i][u]$ for $i \in [0, r)$
8 $\beta_j = tileB[u][t_x + 16j]$ for $j \in [0, r)$
9 $sum[i][j] = sum[i][j] + \alpha_i \beta_j$ for $i, j \in [0, r)$
10 }
11 **BARRIER**()
12 }
13 store $sum[i][j]$ at row $t_y + 16i$, column $t_x + 16j$ of the tile

BARRIER() waits until every thread of the block has reached it. Lines 7 and 8 read $2r = 8$ values from shared memory, and line 9 performs $r^2 = 16$ multiply-adds, so each multiply-add costs $2/r = 0.5$ reads instead of 2. Global traffic falls as well: each phase copies $2 \cdot 64 \cdot 16$ values for $64 \cdot 64 \cdot 16$ multiply-adds, so the product of an $M \times K$ and a $K \times N$ matrix fetches

$$\frac{M}{64} \cdot \frac{N}{64} \cdot \frac{K}{16} \cdot 2 \cdot 64 \cdot 16 = \frac{2MNK}{64}$$

values, a quarter of the $2MNK/16$ needed with 16×16 tiles. The two barriers play the same roles as in the tiled kernel, whose correctness is proved in [20]. On the Orin NX the blocked kernel is 1.8 to 2.3 times faster than the tiled one (Table 6).

8.2 A repeatable embedding gradient

The gradient of the embedding table adds the gradient g_p of each position p into the row of its token z_p. Many positions share a token. With atomic additions, threads would add into a

shared row in an order that varies from run to run, and because floating-point addition is not associative, the result would vary as well. We fix the order with a counting sort.

EMBEDDING-GRADIENT(z, g, V)

Input: the token z_p and the gradient vector g_p of every position p, and the number of rows V of the embedding table.
Output: the gradient G of the embedding table, one row per token.

```
count[v] = |{p : z_p = v}| for v ∈ [0, V) // on the CPU
start[v] = Σ_{u<v} count[u]
list = positions sorted by token, then by position
for each token v and column c, in parallel { // on the GPU
    G[v][c] = Σ g_p[c] over p in list[start[v] .. start[v + 1]), in order
}
```

For the tokens $z = (2, 0, 2, 1)$, the counts are $(1, 1, 2)$, the starts are $(0, 1, 2, 4)$ and the list is $(1, 3, 0, 2)$, so token 2 receives $g_0 + g_2$ in that order. Each entry is summed by one thread in a fixed order, so the result is the same on every run, and a whole training run can be repeated bit for bit on the same device.

9 Experiments

The hardware is described in Table 5 in the appendix. The main corpus is the Song Describer Dataset [18], 706 two-minute Creative Commons excerpts with 1 106 captions written by listeners. Songs are divided between training and evaluation by artist, so that no voice appears on both sides, which leaves 667 songs for training and 39 for evaluation. We report spectral errors as mean absolute differences of natural-log mel energies; one unit corresponds to about 8.7 dB.

Correctness and speed. All 38 tests pass on the Orin NX, and the kernel tests also pass on the Thor. They compare every kernel with a CPU implementation at deliberately awkward sizes, check every gradient numerically, verify the cache theorem, and confirm that a small network can memorise a fixed batch. On the Orin NX, register blocking makes a training step of the first model 1.8 times faster than tiling and 9.3 times faster than the direct kernel (Table 7).

First training runs. Table 1 summarises the runs with the first tokenizer. A run of 1 500 steps reached a held-out loss of 4.93, and a run of 15 000 steps on the Thor reached 4.02. When we restricted the data to the 281 songs whose captions mention a voice, a model fine-tuned from the longer run reached its best held-out loss of 4.23 after only 250 steps, after which its held-out loss rose while its training loss continued to fall; a model trained from scratch on the same songs reached 4.72. The vocal subset appears to be too small for long training.

Table 1: Best held-out loss with the first tokenizer, in nats per code (uniform guess: 6.93).

run	data	steps	loss (nats per code)
first run, Orin NX	all songs	1 500	4.93
longer run, Thor	all songs	15 000	4.02
fine-tuned, Thor	vocal songs	250	4.23
from scratch, Orin NX	vocal songs	5 500	4.72

9.1 What limits the sound

The generated voices sounded pitched but blurred. To find out why, without involving the model, we rendered 12-second excerpts of four held-out vocal songs through each stage of the pipeline and compared the results with their exact spectrograms (Figure 7 and Table 2).

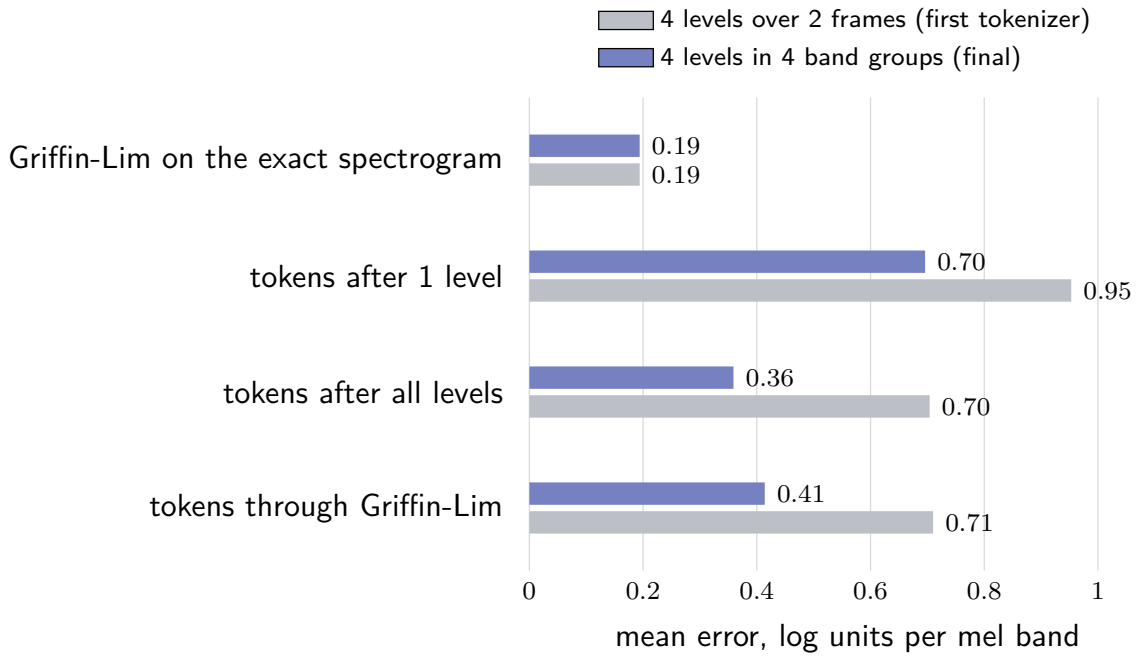


Figure 7: Mean spectral error of each stage on four held-out vocal excerpts, for the first tokenizer (grey) and the final one (blue). The bottom pair is the best that any model trained on these tokens could achieve.

Table 2: Spectral error of each stage on four held-out vocal excerpts, in log units per mel band, for the first tokenizer and the final one.

		first tokenizer		final tokenizer	
excerpt	vocoder	tokens	end to end	tokens	end to end
guitar, brass, female voice	0.230	0.807	0.821	0.425	0.480
folk metal, male voice	0.152	0.707	0.703	0.353	0.378
electronic, vocal sample	0.170	0.569	0.554	0.268	0.338
bass, piano, wordless voice	0.222	0.732	0.763	0.390	0.460
mean	0.194	0.704	0.710	0.359	0.414

With the first tokenizer, the tokens alone caused 3.6 times the error of the vocoder, and the end-to-end error of a perfect model, 0.71, was almost entirely due to the tokenizer. An error of about 0.7 log units, or roughly 6 dB per band, is enough to blur the harmonic and formant detail on which an intelligible voice depends. This measure does not capture the phase artefacts of Griffin-Lim, which remain audible, so the tokenizer is the main spectral limitation rather than the only audible one.

To examine the model on its own, we trained it on a single 12-second excerpt of 562 frames. Its training loss fell to 0.054 after 100 steps and to 0.002 after 200. We then generated 12 seconds from the excerpt's caption and compared the generated codes with the excerpt's own (Table 3). Greedy decoding reproduced all 562 frames, and top-k sampling departed from the excerpt only at isolated codes, without drifting away.

Table 3: Generated codes compared with a memorised excerpt.

decoding	agreement per codebook (%)	identical prefix (frames)
greedy	100.0, 100.0, 100.0, 100.0	562 of 562
top-k, seed 1	99.6, 100.0, 100.0, 99.6	26
top-k, seed 2	100.0, 99.8, 99.3, 99.8	26

The model can reproduce audio exactly, and when it does, the result sounds like the tokenizer's reconstruction. These two observations led us to improve the tokens before training larger models.

9.2 A finer tokenizer

Guided by this analysis, we compared tokenizers that differ in the number of frames per step, the number of band groups and the number of levels (Table 9). Quantising one frame per step in 4 band groups with 4 levels each lowered the mean error on the held-out songs from 0.776 to 0.401. A control without band groups suggests that most of this gain comes from the larger number of codes per step, and a smaller part from the grouping. On the four vocal excerpts of Figure 7, the error of the tokens fell from 0.70 to 0.36, and the end-to-end error from 0.71 to 0.41. The price is a vocabulary of 16 codebooks instead of 4.

9.3 A larger model on the finer tokens

With the finer tokens we began training a larger model on the Thor: 8 blocks of width 512, 42.9 million parameters, with the lyrics input enabled from the start. It is trained on all 742 training songs of the combined corpora, with the lyrics corpus weighted 4-fold and two takes recorded by the first author weighted 10-fold, in batches of 8 sequences of 1 408 positions. A step takes 2.0 seconds and about 11 GB of memory. Training is still in progress; after 4 500 of 20 000 steps the held-out loss is 5.19 nats per code, averaged over 16 codebooks, and still falling. Because the vocabulary differs, this figure is not comparable with those of Table 1.

9.4 Lyrics

We extended both vocal models of Table 1 with a lyrics field and trained them further on vocal songs, JamendoLyrics, and songs whose captions mention rock or a related style. Table 4 gives the result of **Lyrics-Test** for a model trained on the Thor with the lyrics corpus weighted 8-fold, over 48 batches of 4 windows.

Table 4: Loss with the right lyrics, no lyrics and the wrong lyrics, and the paired difference $\bar{\Delta} \pm s$.

songs	right	none	wrong	$\bar{\Delta} \pm s$
		(nats per code)		
held out (6)	4.3345	4.3409	4.3345	$+0.0000 \pm 0.0004$
training (73)	4.0337	4.0395	4.0378	$+0.0042 \pm 0.0006$

On the held-out songs, the right and the wrong lyrics give the same loss, so this model does not yet use the words. On the training songs the difference is statistically clear but small, which

we take to indicate memorisation of those songs rather than a learned relation between words and sound. We hope that the paired test will be useful as a clear criterion for future attempts.

10 Limitations

The tokenizer, although improved, still limits the fidelity of the output, and Griffin-Lim adds phase artefacts that a learned vocoder would remove. The data contain comparatively little singing (298 of the 706 songs mention a voice) and very little of some styles (two songs are described as grunge). Our models are much smaller than published systems [3], and we have not yet carried out a formal listening test, so we make no claims about perceived quality. On the implementation side, all arithmetic uses 32-bit floating point without tensor cores, and attention stores its full matrix of scores [5].

11 Conclusion

We have described StaleyNET, a complete text-to-music system written in Rust and trained from scratch on embedded GPUs. Building every stage ourselves allowed us to measure each one separately, and these measurements pointed to the tokenizer, rather than the language model, as the main limitation on sound quality. A revised tokenizer that quantises groups of frequency bands roughly halved the reconstruction error, and a larger model is now being trained on it. We hope that the worked examples, the stage-by-stage analysis and the paired lyrics test will be of use to others who build similar systems on modest hardware.

References

[1] Andrea Agostinelli et al. MusicLM: Generating music from text, 2023. arXiv:2301.11325.

[2] Zalán Borsos et al. AudioLM: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023. arXiv:2209.03143.

[3] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.05284.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2205.14135.

[6] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023. arXiv:2210.13438.

[7] Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music, 2020. arXiv:2005.00341.

[8] Simon Durand, Daniel Stoller, and Sebastian Ewert. Similarity-based audio-lyrics alignment of multiple languages. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2023. arXiv:2306.07744.

[9] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Annual Meeting of the Association for Computational Linguistics*, 2018. arXiv:1805.04833.

[10] Daniel W. Griffin and Jae S. Lim. Signal estimation from modified short-time Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984.

[11] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016. arXiv:1606.08415.

[12] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021. arXiv:2207.12598.

[13] David B. Kirk, Wen mei W. Hwu, and Izzat El Hajj. *Programming Massively Parallel Processors*. Morgan Kaufmann, 4 edition, 2022.

[14] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2010.05646.

[15] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. High-fidelity audio compression with improved RVQGAN. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.06546.

[16] Jinglin Liu, Chengxi Li, Yi Ren, Feiyang Chen, and Zhou Zhao. DiffSinger: Singing voice synthesis via shallow diffusion mechanism. In *AAAI Conference on Artificial Intelligence*, 2022. arXiv:2105.02446.

[17] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[18] Ilaria Manco, Benno Weck, SeungHeon Doh, Minz Won, Yixiao Zhang, Dmitry Bogdanov, Yusong Wu, Ke Chen, Philip Tovstogan, Emmanouil Benetos, Elio Quinton, György Fazekas, and Juhan Nam. The song describer dataset: a corpus of audio captions for music-and-language evaluation. In *NeurIPS Workshop on Machine Learning for Audio*, 2023. arXiv:2311.10057.

[19] NVIDIA. cuda-oxide: A Rust to PTX compiler backend. https://github.com/NVlabs/cuda-oxide, 2026.

[20] Arpan Pathak. An introduction to tiled parallel programming, simplified, 2026. Manuscript.

[21] Nathanaël Perraudin, Peter Balazs, and Peter L. Søndergaard. A fast Griffin-Lim algorithm. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, 2013.

[22] Reiner Pope et al. Efficiently scaling transformer inference. In *Conference on Machine Learning and Systems*, 2023. arXiv:2211.05102.

[23] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[24] Hubert Siuzdak. Vocos: Closing the gap between time-domain and Fourier-based neural vocoders for high-quality audio synthesis, 2023. arXiv:2306.00814.

[25] Malcolm Slaney. Auditory toolbox, version 2. Technical Report 1998-010, Interval Research Corporation, 1998.

[26] Julius O. Smith. Digital audio resampling home page. Center for Computer Research in Music and Acoustics, Stanford University, 2002. https://ccrma.stanford.edu/~jos/resample/.

[27] Daniel Stoller, Simon Durand, and Sebastian Ewert. End-to-end lyrics alignment for polyphonic music using an audio-to-character recognition model. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2019. arXiv:1902.06797.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.03762.

[29] Vasily Volkov and James W. Demmel. Benchmarking GPUs to tune dense linear algebra. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2008.

[30] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 2020. arXiv:2002.04745.

[31] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. arXiv:2105.13626.

[32] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021. arXiv:2107.03312.

[33] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, 2019. arXiv:1910.07467.

A Hardware and software

Table 5: The two devices. Core counts and memory bandwidth are the manufacturer's specifications; memory capacity, compute capability and software versions were read from the devices.

	Jetson Orin NX 16 GB	Jetson AGX Thor
GPU	NVIDIA Ampere, 1 024 CUDA cores, 32 tensor cores	NVIDIA Blackwell, 2 560 CUDA cores, 96 tensor cores
compute capability	8.7	11.0
CPU	8-core Arm Cortex-A78AE	14-core Arm Neoverse-V3AE
memory	16 GB LPDDR5, shared by CPU and GPU	128 GB LPDDR5X, shared by CPU and GPU
memory bandwidth	102 GB/s	273 GB/s
system software	L4T R36.4.3, CUDA 12.6	L4T R39.2.1, CUDA 13.2
Rust toolchain	nightly-2026-04-03	nightly-2026-10-11
cuda-oxide	revision b7fe543	revision e027ac3

Both devices were shared with other work during our experiments. In particular, language-model services occupied most of the Thor's memory until the larger training run began, at which point they were stopped.

B Benchmarks

All figures in this appendix were measured on the Orin NX with `staleynet bench`. Each timing is the mean of five runs after one warm-up run, and each table entry is the median of three such measurements, because the clock frequency of the device varies noticeably from run to run.

Table 6: Matrix products at the sizes used by the first model, in GFLOP/s.

product ($M \times N \times K$)	direct	tiled	register-blocked	blocked / tiled
attention projections ($2048 \times 768 \times 256$)	46.8	258.6	579.0	2.2
feed-forward ($2048 \times 1024 \times 256$)	55.3	255.5	450.6	1.8
output heads ($2048 \times 4096 \times 256$)	31.2	222.1	458.5	2.1
spectrogram ($4096 \times 513 \times 1024$)	47.7	236.1	541.7	2.3

Table 7: One training step of the first model on 4×512 positions, forward and backward.

matrix-product kernel	time per step (ms)
direct	2 930
tiled	576
register-blocked	315

Table 8: Other measurements on the Orin NX.

measurement	result
log-mel spectrogram of 60 s, GPU	47 ms
log-mel spectrogram of 60 s, CPU (rustfft, 8 threads)	73 ms
decoding step, key/value cache	6.2 ms
decoding step, recomputing the context	17.1 ms

On the Thor, measured during training runs of identical shape, register blocking reduced a training step of the first model on 4×768 positions from 238 ms to 147 ms, and a step of the larger model on $8 \times 1\,408$ positions takes about 2.0 seconds.

C Tokenizer study

Table 9 lists the mean reconstruction error over all 39 held-out songs for the tokenizers we compared, each trained on the same training songs with 1 024 prototypes per codebook. The columns give the number of frames per step, the number of band groups, the number of residual levels in each group, and the resulting number of codes per step.

At 8 codes per step, dividing the bands into groups did not help. With 16 codes per step it helped a little: 16 levels without groups reached 0.427, and 4 groups of 4 levels reached 0.401. Most of the improvement over the first tokenizer therefore comes from describing each step with more codes, and a smaller part, about 6 %, from quantising the band groups separately.

Table 9: Held-out reconstruction error of different tokenizers, in log units per mel band.

frames	groups	levels	codes	error (log units per band)
2	1	4	4	0.776
2	1	8	8	0.643
1	1	4	4	0.745
1	1	8	8	0.588
1	4	2	8	0.597
1	8	1	8	0.612
1	1	16	16	0.427
1	8	2	16	0.403
1	4	4	16	0.401